

Chapter 3

1. Why is a bias added to the actual exponent in the IEEE 754 floating-point representation, and how does this affect the encoding of both positive and negative exponents?

-In IEEE 754 floating-point:

- A **bias** is added to the exponent so it can be stored as an unsigned value.
- This lets the same exponent field represent both **positive** and **negative** exponents.
- **Effect:**
 - Positive exponents → stored as values **above the bias**.
 - Negative exponents → stored as values **below the bias**.

This makes comparisons and hardware implementation simpler.

2. How does optimized multiplication improve efficiency and performance compared to traditional long multiplication, especially in terms of speed and computational complexity?

-Optimized multiplication (like Booth's algorithm) is faster than traditional long multiplication because it:

- Reduces the number of partial products → fewer additions.
- Handles signed numbers efficiently → avoids repeated operations.
- Cuts computational complexity → less shifting and adding.

3.

	Instruction name
1	mul
2	mulh
3	mulhu
4	mulhsu

You must answer the following questions for the explanation of each of the instructions mentioned above:

- (i) What do they do/ How do they work?
- (ii) Explain their syntax.
- (iii) Write an example for each of the instructions.

-Here's the breakdown of the **RISC-V multiplication instructions** from your book:

1. **mul**

(i) What it does:

- Multiplies two 32-bit (or 64-bit) integers.
- Keeps only the **lower half** of the product (least significant bits).

(ii) Syntax:

mul rd, rs1, rs2

- **rd** = destination register
- **rs1, rs2** = source registers

(iii) Example:

`mul x5, x6, x7` # `x5 = (x6 * x7)` lower 32 bits

2. **mulh**

(i) What it does:

- Multiplies two **signed integers**.
- Keeps the **upper half** of the product (most significant bits).

(ii) Syntax:

`mulh rd, rs1, rs2`

(iii) Example:

`mulh x8, x9, x10` # `x8 = upper 32 bits of (signed x9 * signed x10)`

3. **mulhu**

(i) What it does:

- Multiplies two **unsigned integers**.
- Keeps the **upper half** of the product.

(ii) Syntax:

`mulhu rd, rs1, rs2`

(iii) Example:

`mulhu x11, x12, x13` # x11 = upper 32 bits of (unsigned x12 * unsigned x13)

4. **mulhsu**

(i) What it does:

- Multiplies a **signed integer (rs1)** with an **unsigned integer (rs2)**.
- Keeps the **upper half** of the product.

(ii) Syntax:

`mulhsu rd, rs1, rs2`

(iii) Example:

`mulhsu x14, x15, x16` # x14 = upper 32 bits of (signed x15 * unsigned x16)

Summary:

- **mul** → low bits of product.
- **mulh** → high bits (signed × signed).
- **mulhu** → high bits (unsigned × unsigned).
- **mulhsu** → high bits (signed × unsigned).

4.fle.d x5, f3, f5

(i) Why is **x5** an integer register?

- **fle.d** compares two floating-point registers (**f3** and **f5**) and stores the result (true/false) as an **integer 0 or 1** in an integer register.
- That's why the destination is **x5** (an integer register), not a floating-point register.

(ii) **f3** and **f5** both are single precision numbers. True or False?

- **False.**
 - The **.d** in **fle.d** stands for **double precision**.
 - Correct statement: **f3** and **f5** are **double precision (64-bit) floating-point registers**.
-

(iii) After running the instruction, **x5** contains 0. What does this mean?

- **fe.d** sets the destination to 1 if **f3** $\leq$ **f5**, otherwise 0.
 - If **x5** = 0, it means **f3** is greater than **f5** or one of them is NaN (Not-a-Number).
-

So the answers are:

1. Because result is an integer (0/1).
2. False $\rightarrow$ They are **double precision**, not single.
3. **f3** > **f5** or comparison was unordered due to NaN.

5. Write the necessary RISC-V code to compare two floating point registers f1 and f2. If f1 and f2 are equal jump to a label called “jumpEqual” else jump to another label called “jumpNotEqual” Note: both the numbers stored in f1 and f2 registers are double-precision floating point numbers.

```
feq.d x5, f1, f2    # x5 = 1 if f1 == f2 else 0
```

```
bne x5, x0, jumpEqual # If x5 != 0  $\rightarrow$  f1 == f2  $\rightarrow$  go to jumpEqual
```

```
beq x5, x0, jumpNotEqual # If x5 == 0  $\rightarrow$  f1 != f2  $\rightarrow$  go to jumpNotEqual
```

6. Write the necessary RISC-V code to compare two floating point registers f1 and f2. If f1 is greater than f2, jump to a label called “jumpGreater” else jump to another label called “jumpNotGreater” and divide the variable c with 16 and store that answer in c. Note: both the numbers stored in f1 and f2

registers are double-precision floating point numbers and variable c is stored in x19.

```
fgt.d t0, f1, f2
```

```
bnez t0, jumpGreater
```

```
jumpNotGreater:
```

```
srli x19, x19, 4    # c = c / 16
```

```
jumpGreater:
```

Chapter 4

1. Provide a detailed explanation of how the register file performs both read and write operations.

-A register file is a collection of registers that can be read from and written to by specifying a register number.

- **Read operation:** To perform a read, the register file takes a register number as input and outputs the data stored in that register. An output signal carries the read value.
- **Write operation:** To perform a write, three inputs are needed: the number of the register to be written, the data to be written into the register, and a clock signal that controls when the write occurs.

- **Simultaneous read and write:** In an edge-triggered system, a register can be both read and written in the same clock cycle. The read operation retrieves the value written in a previous clock cycle, while the new value is written at the clock edge and becomes available for a read in the subsequent clock cycle.

2.Explain why the size of the read/write register pin is 5 bits.

-The size of the register number pin is 5 bits because the RISC-V architecture specifies that there are 32 general-purpose registers. In binary, 32 can be represented using 5 bits, as $2^5 = 32$. This is a common design choice in RISC architectures like MIPS and RISC-V to balance the need for a sufficient number of registers with the constraints of instruction encoding.

3. Explain the significance of the Zero pin in the ALU.

-The Zero pin on the ALU is a crucial status signal that is asserted (set to 1) when the result of an ALU operation is zero. It is essential for implementing conditional branch instructions like **beq** (branch if equal). For a **beq** instruction, the ALU performs a subtraction of two operands. If the operands are equal, the result is zero, and the Zero pin is asserted. The processor's control unit then uses this signal to decide whether to update the program counter (PC) with the branch target address or continue with the next sequential instruction, thereby changing the program's control flow. This simple mechanism enables the processor to make decisions in its hardware logic.

4. Describe how the ALU determines which operation to perform on the given Inputs.

-The ALU is a combinational circuit that determines its operation based on a control signal from the control unit. This process starts when the CPU fetches and decodes an instruction that contains an **opcode** and the **operands** to be used. The control unit then sends the operands to the ALU and applies the opcode to a dedicated control input. The ALU immediately executes the specified arithmetic or logical function, and its output becomes available after the signals have propagated

through the circuit. The number of distinct operations an ALU can perform is limited by the bit-width of its control input.

5.Does the ALU control utilize Instruction bits 30 and 14-12 to generate the output for the LD instruction? Provide a justification for your answer.

-No, the ALU control does not use instruction bits 30 and 14-12 to generate the output for a **LD** (load) instruction.

Justification

Based on the provided document, the ALU control uses instruction bits 30 and 14-12 only in the specific case where the **ALUOp** signal is 10. This **ALUOp** value is exclusively for **R-type instructions** (like **add**, **sub**, **and**, and **or**) that require the ALU control to look at the function fields to determine the specific operation.

The **LD** instruction is an **I-type instruction**, not an R-type instruction. For I-type instructions, the ALU's operation is dictated directly by the **ALUOp** signal, which would not be 10 in this case. Therefore, the ALU control does not need to inspect instruction bits 30 and 14-12 for the **LD** instruction.

6.Identify the specific cases in which the ALU control utilizes Instruction bits 30 and 14-12 to generate the output. Additionally, explain why only these four bits (Instruction bits 30 and 14-12) are used in such cases.

-The ALU control unit determines the specific operation for R-type instructions by using a combination of the **ALUOp** control signal and specific bits from the instruction's function fields (**funct7** and **funct3**). When the main control unit sets **ALUOp** to 10, it signals the ALU control to look at these function fields for the specific operation. For R-type instructions like **add**, **sub**, **and**, and **or**, only four bits are necessary to differentiate between them: bit 30 from the **funct7** field and bits 14-12 from the **funct3** field. Using only these four bits simplifies the control logic, reducing hardware complexity and latency.

7. What do you understand by the term "single-cycle" in the context of a single-cycle datapath?

-In the context of a single-cycle datapath, the term "**single-cycle**" refers to the fact that every instruction is fetched, decoded, executed, and completed in a **single clock cycle**.

This design simplifies the processor's control logic because each instruction follows the same path and timing, eliminating the need for complex state management or inter-stage coordination. However, a major drawback is that the duration of this single clock cycle must be long enough to accommodate the absolute slowest instruction in the instruction set, such as a load or store instruction. This means that simpler and faster instructions, like **add** or **and**, must also wait for the entire, long clock cycle to complete, which limits the overall performance and throughput of the processor.

8. Write a comparison between the single cycle datapath and a pipelined datapath. Mention why pipelined datapath is implemented in real life.

Feature	Single-Cycle Datapath	Pipelined Datapath
Execution	One instruction at a time	Multiple instructions at a time (one per stage)
Clock Cycle	Long, determined by the slowest instruction	Short, determined by the longest pipeline stage
Throughput	Low (one instruction per clock cycle)	High (ideally, one instruction completed per clock cycle)
Hardware	Less complex, no duplication of hardware for different stages	More complex, requires duplicate hardware for stages that run concurrently
Latency	Low latency for a single instruction	High latency for a single instruction due to the delay of passing through all stages

Why Pipelining is Used in Real Life

Pipelining is a fundamental technique used in almost all modern processors because it significantly increases **throughput** and overall performance. While the time it takes to complete a single instruction (latency) might be slightly longer due to the added registers between stages, the ability to finish one instruction per clock cycle dramatically improves the number of instructions executed per second. This parallel execution of instructions makes pipelining far more efficient for running programs in the real world, despite the added complexity of managing hazards and dependencies between instructions.

9. In the RISC-V pipelined datapath we have 5 stages. Can we divide these 5 stages into more stages to make the datapath more efficient?

-Yes, you can divide the 5 stages of a RISC-V pipelined datapath into more stages to make it more efficient. This process is known as deeper pipelining.

Deeper pipelining, which involves dividing the five stages of a RISC-V datapath into more, smaller stages, can make a processor more efficient. This is because splitting the stages reduces the work each one does, allowing for a higher clock frequency and thus increased throughput. However, this process has significant drawbacks. The latency for a single instruction to complete increases, and the complexity and likelihood of pipeline hazards (like data and control hazards) grow. Deeper pipelining also leads to increased power consumption and a larger datapath due to the need for more pipeline registers and control logic.

10. Explain Why do data hazards occur in a pipelined Data path?

-Data hazards happen because a pipelined datapath allows multiple instructions to be processed at the same time, but an instruction in a later stage might need the result of a previous instruction that hasn't finished yet. Specifically, an instruction might try to read a register before a preceding instruction has written its new value to that same register. This creates a dependency where the later instruction gets an old, incorrect value. To prevent this, the pipeline has to either stall or use forwarding, which adds complexity but ensures correct execution.

11. What do you understand about structural hazards? Explain with an example.

-A structural hazard occurs when two instructions in a pipeline try to use the same hardware resource at the same time. This resource conflict forces one instruction to wait, which stalls the pipeline and reduces performance.

For example, in a simple 5-stage pipeline with a single memory, a structural hazard occurs if a **load** instruction is in its Memory Access (MEM) stage while the next instruction is in its Instruction Fetch (IF) stage. Both stages need to access the memory at the same time: one to read data, the other to fetch the next instruction. Since the single memory port can only handle one request at a time, the pipeline must stall, preventing the second instruction from proceeding.

Prepared by Azmari Sultana